

# Computer Architecture, Organization, and Performance

Week 1 — How Fast Is This Machine, Really?

**Dr. Auqib Hamid Lone**

Assistant Professor in Computer Science & Engineering

**PCC CS-401: Computer Organization and Architecture**

Department of Computer Science & Engineering  
Government College of Engineering & Technology (GCET), Ganderbal

August 7, 2026

# Welcome: What This Course Is

## The big question

A program is written in a high-level language, compiled to machine instructions, and run on hardware. What sits *between* the program and the silicon — the **machine** that fetches, decodes, and executes?

- ▶ **Computer organization**: how the hardware is built — registers, ALU, control unit, buses, memory.
- ▶ **Computer architecture**: what the programmer sees — the instruction set, addressing modes, data types.
- ▶ Together they decide *how fast* a program runs and *what* it can do.

# Roadmap: Four Questions, One Thread

1. **Architecture vs. organization** — what is the difference, and why does it matter?
2. **Basic structure** — what are the parts of a computer, and how are they connected?
3. **The performance equation** — what determines execution time?
4. **Amdahl's Law** — when you speed up one part, how much faster does the whole machine get?

## Running thread for today

We will compare two machines running the *same* program: count instructions, measure clock rate and CPI, compute execution times — then ask how much a proposed hardware upgrade actually helps.

Before we measure speed, we need to know what a computer is —  
and what makes two computers different.

# Architecture vs. Organization: The Interface and the Implementation

## Computer architecture (the **interface**)

What the programmer/compiler sees: the instruction set, registers, data types, addressing modes. Two machines with the same architecture run the same binary (e.g. different x86 models).

## Computer organization (the **implementation**)

How the hardware realizes that interface: the datapath, control unit, bus structure, memory hierarchy. Two machines with the same architecture can differ wildly in organization — and in speed.

Same architecture  $\Rightarrow$  same programs run. Different organization  $\Rightarrow$  they run at different speeds and costs.

# Why the Distinction Matters

- ▶ **Compatibility:** keeping the architecture stable means old software keeps running on new hardware — the whole reason x86 has survived for decades.
- ▶ **Design freedom:** within one architecture, engineers are free to redesign the organization (pipelining, caches, faster ALUs) to make the same instructions execute faster.
- ▶ **The course arc:** Weeks 4–7 build the organization (registers, datapath, control) that implements a fixed instruction-set interface.

## Patterson & Hennessy's framing

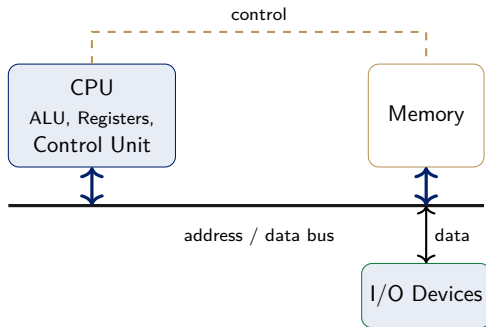
Architecture = the “what” (the programmer’s contract); organization = the “how” (the hardware’s construction) — P&H, *Computer Organization and Design*, ARM Edition, Sec. 1.1, p.3 [2].

# Classes of Computers

- ▶ **Personal computers**: one user, general purpose — performance and cost balanced.
- ▶ **Server computers**: many users / large workloads — availability and throughput matter most.
- ▶ **Supercomputers**: the fastest machines for the largest scientific problems — peak floating-point performance.
- ▶ **Embedded computers**: inside appliances, cars, phones — cost and power dominate; performance is just enough.
- ▶ **Personal mobile devices**: smartphones/tablets — the most common computers today, constrained by power and size.

The design goal differs by class: what is “best” for a phone is not what is “best” for a supercomputer. (Hamacher, Ch. 1; P&H, Sec. 1.1, p.3) [1, 2]

# The Basic Structure of a Computer



The classic three-part structure: **CPU** (control unit + ALU + registers), **memory**, and **I/O**, connected by buses — cf. P&H, Sec. 1.4, “Under the Covers,” p.16 [2].



# What Actually Runs: The Instruction Cycle Preview

## The machine's heartbeat

The CPU repeatedly **fetches** an instruction from memory, **decodes** it, and **executes** it — the instruction cycle. Week 4 dissects this in detail; today we just need the consequence:

- ▶ Execution time is built from *how many instructions* run (IC), *how many cycles each* takes (CPI), and *how fast a cycle is* (clock rate  $f$ ).
- ▶ Those three quantities are the entire subject of the performance equation — next act.

The structure is fixed. Now: how do we measure how fast it runs?

# Motivation: What Does “Faster” Mean?

## Socratic question

Two machines run the same program. One has a faster clock, the other executes fewer instructions per program. Which is “faster” — and how do you decide without guessing?

- ▶ The only honest measure is **time**: execution time of the actual program.
- ▶ Everything else (clock rate, CPI, instruction count) is a *component* of time — useful for *design*, not as a standalone claim of speed.
- ▶ So we need one equation that puts the three components together.

## Definition: Execution Time and Its Three Factors

### Basic performance equation

CPU time = instruction count  $\times$  CPI  $\times$  clock cycle time =  $IC \times CPI \times T_c$ ,

or, with clock rate  $f = 1/T_c$ :

$$\text{CPU time} = \frac{IC \times CPI}{f}.$$

- ▶ **IC** (instruction count): how many instructions the program executes — set by the program + compiler + ISA.
- ▶ **CPI** (cycles per instruction): average clock cycles per instruction — set by the organization (datapath, control).
- ▶  $T_c$  (clock cycle time) =  $1/f$ : how long one cycle takes — set by the clock rate of the hardware.

(P&H, Sec. 1.6, p.28) [2]

## Who Controls Each Factor?

| Factor   | What it measures         | Influenced by               | Changed by          |
|----------|--------------------------|-----------------------------|---------------------|
| IC       | instructions per program | ISA, compiler, program      | architecture        |
| CPI      | cycles per instruction   | datapath, pipeline, memory  | organization        |
| $f(T_c)$ | clock rate               | fabrication, circuit design | technology + design |

The three factors are **independent**: improving one does not automatically improve the others — and sometimes improving one hurts another (e.g. a more complex instruction reduces IC but raises CPI).

## Worked Example: Two Machines, One Program

### The comparison

Program P executes 10 million instructions. Machine A runs at  $f = 2$  GHz with  $\text{CPI} = 2$ ; Machine B runs at  $f = 3$  GHz with  $\text{CPI} = 3$ . Which is faster for P, and by how much?

### Compute both times

A:  $\text{IC} = 10^7$ ,  $\text{CPI} = 2$ ,  $T_c = 0.5\text{ns} \Rightarrow \text{time} = 10^7 \times 2 \times 0.5\text{ns} = 10\text{ms}$ .

B:  $\text{IC} = 10^7$ ,  $\text{CPI} = 3$ ,  $T_c = \frac{1}{3}\text{ns} \Rightarrow \text{time} = 10^7 \times 3 \times \frac{1}{3}\text{ns} = 10\text{ms}$ .

Tie! Despite the faster clock, B's higher CPI cancels it. Faster clock  $\nRightarrow$  faster machine — the equation decides.

## Worked Example: Breaking the Tie

Same setup, but now

Machine B's designer reduces CPI from 3 to 1.5 (better pipelining). Recompute.

Result

B:  $\text{CPI} = 1.5$ ,  $T_c = \frac{1}{3}\text{ns} \Rightarrow \text{time} = 10^7 \times 1.5 \times \frac{1}{3}\text{ns} = 5\text{ms}$ .

Takeaway

Now B is **2× faster** — the improvement came from **CPI** (organization), not from clock rate. This is why the course spends Weeks 5–7 on datapath and control design: they directly set CPI.

## Socratic Check: Which Component Should You Improve?

### Question

A program spends its time entirely on arithmetic. You can either (a) double the clock rate, or (b) halve the CPI of arithmetic instructions. Assuming the rest stays fixed, which gives the bigger speedup — and does it depend on the program?

- ▶ Both improve the same term ( $\text{time} \propto \text{CPI}/f$ ) by  $2\times$  — a *tie* for this program.
- ▶ But real programs mix instruction types: improving only the CPI of *one* type helps only the fraction of time that type occupies. That leads directly to **Amdahl's Law** (Act 4).



We can compute time. Now: how do we measure the factors, and what are the common mistakes?

# What Do We Measure?

## The right measure

**Execution time** of a real, representative workload — seconds (or its inverse, **throughput**: work done per second).

- ▶ **Benchmarks**: standard programs used to compare machines — the more representative of real workloads, the more believable the comparison.
- ▶ **Wall-clock time**: everything counts (memory, I/O, OS) — best for user-visible speed.
- ▶ **CPU time**: only the time the CPU is computing on the program — best for isolating processor design differences.

# MIPS: A Metric to Be Careful With

## Definition

$$\text{MIPS} = \frac{\text{instruction count}}{\text{execution time} \times 10^6} = \frac{f}{\text{CPI} \times 10^6}.$$

Million Instructions Per Second.

## Why it is misleading

- ▶ Ignores what each instruction *does*: a machine that runs a harder instruction set can have more useful work per MIPS.
- ▶ MIPS varies with the program (a program with more easy instructions gets a higher MIPS number).
- ▶ Two machines can report the *same* MIPS but very different execution times for the same program.

## Worked Example: The MIPS Trap

### Setup

Machine A: 1 GHz, CPI 2.0, runs program P with  $IC = 10^6$ . Machine B: 1 GHz, CPI 1.0, runs the *same* P.

### Compare MIPS and time

$$A: \text{MIPS} = \frac{10^9}{2 \times 10^6} = 500, \text{ time} = \frac{10^6 \times 2}{10^9} = 2\text{ms}.$$

$$B: \text{MIPS} = \frac{10^9}{1 \times 10^6} = 1000, \text{ time} = \frac{10^6 \times 1}{10^9} = 1\text{ms}.$$

Here B's MIPS is  $2\times$  and time is  $2\times$  better — consistent. The trap appears only when the instruction *sets* differ; then MIPS numbers become incomparable. Time is always safe.

## Socratic Check: Why Not Just Count Instructions?

### Question

Machine C executes 500 million instructions of program P in 1 second; Machine D executes 800 million instructions of the same P in 1.6 seconds. Which is faster?

- ▶ Compute time:  $C = 1$  s;  $D = 1.6$  s. **C is faster** — despite running *fewer* instructions.
- ▶ Instruction count alone is meaningless without time — **IC** is a component of the equation, not a speed metric on its own.

The equation and its pitfalls are set. Now the most useful tool:  
how much does a partial speedup help?

## Motivation: Speeding Up One Part

### Socratic question

A program takes 100 s. You can make the multiply part  $10\times$  faster, but multiplies are only 20% of the time. How much faster does the whole program get —  $10\times$ ? Obviously not. What is the real number?

- ▶ The unimproved 80% still takes its full time — it cannot shrink.
- ▶ The improved 20% shrinks from 20 s to 2 s.
- ▶ New total:  $80 + 2 = 82$  s, speedup  $\approx 1.22\times$ . Far from  $10\times$ .

# Amdahl's Law

## Statement

If a fraction  $f$  of execution time can be sped up by a factor  $S_f$ , the overall speedup is

$$\text{Speedup} = \frac{1}{(1 - f) + \frac{f}{S_f}}.$$

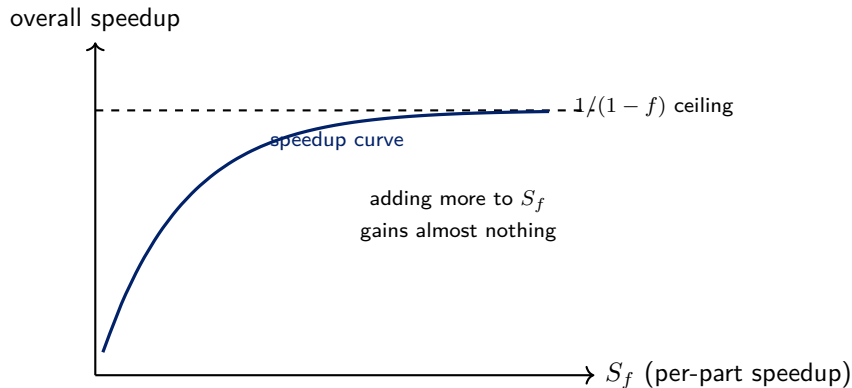
The unimproved fraction  $(1 - f)$  runs at full speed forever; only the improved fraction  $f/S_f$  shrinks.

- ▶ As  $S_f \rightarrow \infty$ , speedup  $\rightarrow 1/(1 - f)$  — a **hard ceiling** set by the unimproved fraction.
- ▶ Improving a part that is only a small fraction of time buys almost nothing, no matter how fast you make it.

(P&H, Sec. 1.6, p.28; Amdahl's Law) [2]



# Amdahl's Law: The Ceiling



The curve saturates: even an *infinite* speedup of the improved part cannot beat the unimproved fraction.

## Worked Example: Applying Amdahl's Law

### Setup

40% of a program's time is spent on floating-point. A new FP unit makes that part 4× faster. Overall speedup?

### Plug in

$f = 0.4$ ,  $S_f = 4$ :

$$\text{Speedup} = \frac{1}{(1 - 0.4) + 0.4/4} = \frac{1}{0.6 + 0.1} = \frac{1}{0.7} \approx 1.43 \times .$$

Even a 4× FP speedup on a 40% fraction buys only 1.43× overall — the 60% unimproved part dominates.

## Worked Example: The Ceiling in Action

### Setup

Same program, 40% FP. How much overall speedup is possible *at best*, i.e. as the FP part becomes infinitely fast?

### Ceiling

$$\lim_{S_f \rightarrow \infty} \frac{1}{(1-f) + f/S_f} = \frac{1}{1-f} = \frac{1}{0.6} \approx 1.67 \times .$$

### Design implication

If you want a program to run  $3\times$  faster and FP is only 40%, the FP unit is the *wrong place to invest* — even infinite FP speedup cannot reach  $3\times$ . You must also speed up the other 60%.

# Socratic Check: The Parallelism Twist

## Question

Amdahl's Law is often quoted against parallel computing: “even with infinite processors, the serial fraction limits speedup.” Is that right, and what is the one assumption that makes it optimistic or pessimistic?

- ▶ Right: serial fraction  $(1 - f)$  caps speedup at  $1/(1 - f)$ .
- ▶ The assumption:  $f$  is **fixed**. If the problem *grows* (larger  $n$ ), the parallel fraction usually grows with it — so larger problems extract more parallelism, softening the ceiling. Amdahl's Law is a statement about *fixed-size* workloads.

One equation, three components, and a law about limits. Now: put it all together.

# The Thread, Closed

## What we built today

- ▶ **Time** is the only honest speed measure:  $\text{time} = \text{IC} \times \text{CPI} \times T_c$ .
- ▶ **IC** (architecture), **CPI** (organization),  $f$  (technology) — three independent levers.
- ▶ **MIPS** misleads when instruction sets differ; time never does.
- ▶ **Amdahl's Law** tells you which lever is worth pulling — and which is not.

## The running comparison

Machine A (2 GHz, CPI 2) and Machine B (3 GHz, CPI 3) tied at 10 ms for a  $10^7$ -instruction program; improving B's CPI to 1.5 made it  $2\times$  faster — a CPI (organization) win, not a clock-rate win.

# Bridge to Week 2

## What's still missing

The performance equation assumes we can *count instructions and CPI*. But before the CPU can execute anything, the **data** it works on must be represented in bits — and arithmetic must be implemented in hardware.

## Next week

**Number systems:** how integers are represented (two's complement) and how adders/subtractors implement arithmetic in hardware — the “what the bits mean” layer that the instruction set operates on.

# Summary

- ▶ **Architecture** = the programmer's interface; **organization** = the hardware implementation. Same architecture, different organization, different speed.
- ▶ **Basic structure**: CPU (ALU + registers + control) connected to memory and I/O by buses.
- ▶ **Performance equation**:  $\text{CPU time} = \text{IC} \times \text{CPI} / f$  — three independent factors.
- ▶ **Measurement**: time (or throughput) on a representative benchmark; beware MIPS.
- ▶ **Amdahl's Law**:  $\text{Speedup} = 1 / [(1 - f) + f / S_f]$  — the unimproved fraction sets an unbreakable ceiling.



# References



V. Carl Hamacher, Zvonko G. Vranesic, and Safwat G. Zaky.

*Computer Organization.*

McGraw-Hill, 5th edition, 2002.



David A. Patterson and John L. Hennessy.

*Computer Organization and Design: The Hardware/Software Interface.*

Morgan Kaufmann, arm edition (5th) edition, 2017.